

Homework 1: Instruction-Level Parallelism

Problem 1. Write instruction-level pseudocode that uses a Taylor series to calculate e^x and provide a dependency graph for the loop.

```
float exp(float x, int n) {
```

Dependency Graph:

Problem 2. The following function calculates the ℓ_2 norm (Euclidean norm) of an n -dimensional vector. Assume that each numbered line represents an operation taking a single clock cycle. The first loop (line 02) is completely independent across all n values of x and can be parallelized. The others must be run in serial. What percentage of this program is parallel? According to Amdahl's law, what speedup can we expect if both n and the number of processors are large?

```

float l2(float* x, int n) {
01   int i = 0;
      while(i < n) {
02       x[i] = x[i] * x[i];
      }
03   i = 0;
04   float sum_sq = 0;
      while(i < n) {
05       sum_sq = sum_sq + x[i]
      }
06   return sqrt(sum);
}

```